Meta-harness on Islo

Deterministic simulation: 0/5 to 5/5 in four proposer steps

Yossi Eliaz Incredibuild · Islo yossi.eliaz@incredibuild.com · yossi@islo.dev

2026-05-05 · revised 2026-10-22

An offline orchestration prototype inspired by Yoonho Lee's meta-harness, with an illustrative Islo integration stub.

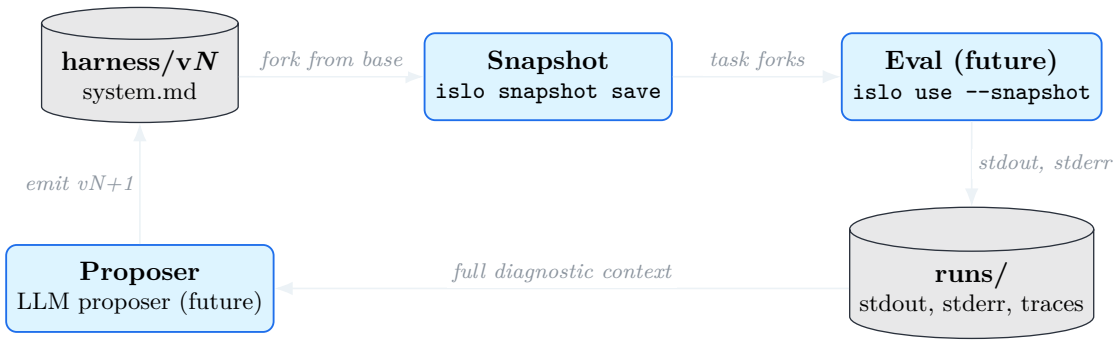
Scope: All reported scores come from a deterministic Python agent simulator and a hardcoded task-hint proposer. No LLM or Islo sandbox was used for these results. The five tasks are designed fixtures, not a held-out benchmark.

The idea, in one paragraph

A *harness* is the prompt + tools + scaffolding around an LLM agent. A *meta-harness* is the loop that improves the harness automatically: a proposer agent reads the diagnostic logs of prior candidates, spots a failure mode, and writes a better harness. The useful design question is what diagnostic context the proposer receives. This prototype explores the *eval–inspect–revise* control flow with a simulator and a predefined hint table.

A real implementation needs a runtime that can produce, store, and serve those traces. This prototype explores the artifact contracts a sandbox-backed implementation would need; it does not validate Islo behavior.

Proposed Islo architecture



Three things meta-harness needs from its runtime:

1. **Reproducible eval environments** — every candidate harness runs against the *same* setup, otherwise the score is noise.
2. **Massive parallelism** — testing N candidates × K tasks adds up fast.
3. **Persistent traces** — the proposer needs to read stdout/stderr/agent-thoughts from runs that completed an hour ago.

Proposed CLI mapping (not exercised by the simulation):

```
islo snapshot save meta-base                  # prepare environment once
islo use mh-cand-7 --snapshot meta-base ...   # fork per candidate, in parallel
islo logs mh-cand-7 --type agent              # diagnostic traces, durable
```

These commands illustrate a proposed integration; the simulation does not exercise them. Source hydration, output capture, gateway policy, and CLI compatibility need validation. A Harbor workload adapter is not included.

The POC

```
tasks/        # 5 exact-match toy tasks, each prompt.md + grade.sh
harness/v0/   # baseline system prompt, deliberately mediocre
bin/
  meta-harness    # bash orchestrator (eval | propose | loop | viz)
  agent-sim.py    # deterministic agent stand-in (offline mode)
  proposer.py     # reads runs/, emits harness/vN+1
viz/index.html    # live dashboard
runs/             # populated each iteration
```

Five tasks: FizzBuzz 1–15, first 10 primes, reverse a list, sum even numbers in [1..10], "is racecar a palindrome." Each grades by exact-match on stdout. Each has a known failure mode in v0 (off-by-one, includes 1 as prime, comma vs space separator, exclusive vs inclusive bounds, wrong case).

The agent is a Python simulator that's intentionally buggy — until the system prompt contains the right hint keyword. So the loop is *deterministic and offline*. A real Claude/Islo backend still needs implementation and validation.

The proposer reads `runs/iter-N/`, finds which tasks failed, looks up the missing hint for that task, and appends it to a new `harness/v{N+1}/system.md`. A real proposer would be:

```
islo use --snapshot meta-base --agent claude --task "
  Examine /workspace/runs/iter-${N}. Find a common failure mode in the
  grade.sh stderr. Write /workspace/harness/v${N+1}/system.md as a small
  edit on top of v${N}/system.md to fix it."
```

This is a proposed input/output contract. The Islo path is an unvalidated stub with a placeholder repository and output capture; the current evaluator runs tasks sequentially.

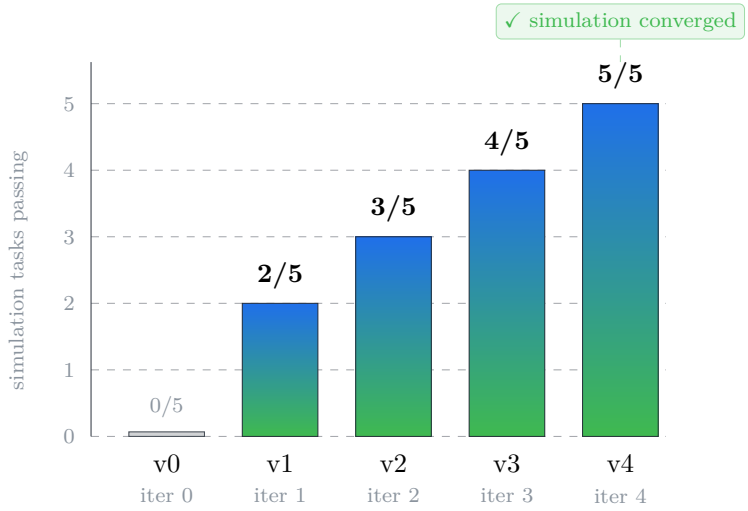
Designed keyword overlap

In simulation iteration 1, fizzbuzz's *inclusive* hint also triggers sum-evens, moving the score from 0/5 to 2/5. This designed keyword overlap does not demonstrate LLM transfer learning.

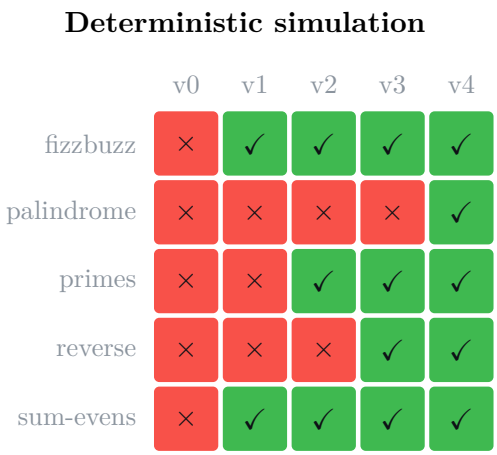
Deterministic simulation results

These scores describe predefined keyword-triggered behavior, not real-agent accuracy or generalization.

Simulation: passing tasks per iteration



Task × iteration heatmap



What changed at each simulation step

iter	harness	score	what changed
0	v0	**0 / 5**	baseline — minimal “produce only the answer” prompt
1	v1	**2 / 5**	+ “Loops are 1-indexed: count from 1 through N inclusive.”
2	v2	**3 / 5**	+ “The smallest prime is 2 — 1 is not prime.”
3	v3	**4 / 5**	+ “When asked for separated values, use space-separated output.”
4	v4	**5 / 5**	+ “Format constraints are exact-case — produce all lowercase only when required.”

Five simulation iterations. Four proposer steps. Hard cap was 10. The proposer reads pass/fail

status and selects from a hardcoded task-specific hint table. The intended fixes are encoded; they are not inferred from traces.

The visualization

A single static HTML page (`viz/index.html`) polls `runs/state.json` every 2 seconds and shows three things:

1. **Pass-rate timeline** — bars for each iteration, height = passing tasks.
2. **Task × iteration heatmap** — green/red grid, one cell per (task, harness). The shape of the green wave shows which tasks were fixed in which order, and whether any later iteration regressed an earlier pass.
3. **Trace inspector** — click any cell, see that run's stdout and the `grade.sh` stderr. The current proposer uses pass/fail flags; the viewer exposes richer diagnostics for inspection.

You can also click a timeline bar to see the system prompt at that iteration, with the newly-added section highlighted. This shows the exact predefined rule added at each step.

Dashboard layout (sketch)



What this points at

This POC is intentionally tiny so the loop is observable in one screen. Three obvious next steps:

- **Real backend.** Complete and validate the `BACKEND=islo` stub: source hydration, task/harness synchronization, output capture, and the CLI contract. Parallel task execution is not implemented. No real-backend latency is measured here.
- **Real workload.** Add and validate a Harbor adapter for the `prompt.md` + `grade.sh` task shape.
- **Real proposer.** Replace the predefined hint table with an LLM that reads `runs/`. Evaluate on tasks whose fixes are not encoded in the proposer before claiming generalization.

The working artifact is the local simulation and dashboard. Real-agent results require a separate evaluation.

Code: `bin/meta-harness demo` to run it; `bin/meta-harness viz` to watch it.

zozo123.github.io/meta-harness-on-islo-page · github.com/zozo123/meta-harness-on-islo

Proposed runtime: islo.dev

Yossi Eliaz · Incredibuild · Islo

yossi.eliaz@incredibuild.com · yossi@islo.dev